



Exp :13 Write the python program to implement Minimax algorithm for gaming

Aim:

To write a python program for vacuum cleaner problem.

Algorithm:

Step1: Start from the root node of the game tree at depth 0 with the Maximizer's turn.

Step2: If the current depth reaches the target terminal depth, return the utility score of that leaf node.

Step3: If it is the Maximizer's turn, recursively evaluate the left and right child nodes and return the maximum of the two scores.

Step4: If it is the Minimizer's turn, recursively evaluate the left and right child nodes and return the minimum of the two scores.

Step5: Alternating between Maximizer and Minimizer at each level, propagate the optimal values up the tree to determine the best possible score at the root node.

Program:

```
import math
```

```
def minimax(depth, node_index, is_max, scores, target_depth):
```

```
    if depth == target_depth:
```

```
        return scores[node_index]
```

```
    if is_max:
```

```
        return max(
```

```
            minimax(depth + 1, node_index * 2, False, scores, target_depth),
```

```
            minimax(depth + 1, node_index * 2 + 1, False, scores, target_depth)
```

```
        )
```

```
    else:
```

```
        return min(
```

```

        minimax(depth + 1, node_index * 2, True, scores, target_depth),
        minimax(depth + 1, node_index * 2 + 1, True, scores, target_depth)
    )

if __name__ == '__main__':
    leaf_scores = [3, 5, 2, 9, 12, 5, 23, 23]

    tree_depth = int(math.log2(len(leaf_scores)))

    optimal_value = minimax(0, 0, True, leaf_scores, tree_depth)

    print("Leaf node scores:", leaf_scores)
    print("The optimal value for Maximizer is:", optimal_value)

```

OUTPUT:

```

Leaf node scores: [3, 5, 2, 9, 12, 5, 23, 23]
The optimal value for Maximizer is: 12

```

Result: Thus, the program was executed and found that the output was found to be correct.